Learning Initialisation Heuristic for Large Scale Vehicle Routing Problem with Genetic Programming

Joao Guilherme Cavalcanti Costa, Yi Mei, Mengjie Zhang
School of Engineering and Computer Science
Victoria University of Wellington, PO Box 600,
Wellington, 6140, New Zealand
{Joao.Costa, Yi.Mei, Mengjie.Zhang}@ecs.vuw.ac.nz

***Abstract*—The Large Scale Vehicle Routing Problem is a classical NP-hard problem. It has several applications in the industry and has always been the focus of studies and development of new, ever more complex, techniques to solve it. An important group of these techniques are Local Search-based, which are sensitive to the initial solution given to them. However, finding effective initial solutions is not a trivial task, requiring domain knowledge for building them. Although some Genetic Programming Hyper-Heuristics (GPHH) have tried to build better heuristics automatically, they barely give an advantage for improving the solution afterwards. This paper aims to show that Genetic Programming can identify better regions of the search space, where the initial solutions can be improved more efficiently with optimisation steps. This is done by developing new terminals and a new fitness function, which are based on the width of the routes, a metric that was recently found to be an important feature for good solutions. The obtained results show that the proposed approach finds better final solutions than when using classical initial heuristics or other GPHH, for both time efficiency and effectiveness.**

***Index Terms*—Large Scale Vehicle Routing, Genetic Programming, Hyper-Heuristic, Initialisation**

I. Introduction

The Large Scale Vehicle Routing Problem (LSVRP) is a special case of the Capacitated Vehicle Routing Problem (CVRP) when the number of customers is at least 200, as defined by several studies, such as [1]–[4]. The CVRP is commonly used for determining the delivery or pick-up of goods to customers by a set of vehicles with limited capacity. The problem can be applied to different real-world scenarios, such as mail delivery [5], waste collection [6], among others. The LSVRP faces many challenges due to the large number of possible solutions and is still fairly unexplored compared to smaller-scale benchmark instances, even though a large scale represents a more realistic scenario for a lot of companies. Even successful heuristic methods for the CVRP (such as [7], [8]) require a significant amount of time for solving several of the instances available in literature datasets, as shown in [4].

One reason for this lack of efficiency could be tracked to the huge solution search space of the problem. Most works would find success by doing a faster search or utilising elaborated pruning techniques to reduce the size of the search space, which allows the finding of a better solution more quickly. In other words, they would focus on improving the existing methodologies (mostly the Local Search-based), rather than understanding the solution space.

Although understanding how the search space looks like can be seen as a hard task, a recent work, by [9] (reinforced by [10]) was able to move in the right direction. They have presented some insight and data analysis on the VRP solution characteristics. These works present some metrics which are common in the better solutions for the VRP. In [11], they utilised the extracted information to build a Knowledge-Guided Local Search (KGLS), finding state-of-the-art results for the large and very large scale VRP. The success of the algorithm proves that the analysis is indeed valid and this information can be used to search more effectively the solution search space. However, their algorithm is still prone to the quality of the initial solution search space, requiring several penalisation runs (i.e., jumping out of the local optima) to find a region with better possible solutions.

Additionally, creating efficient algorithms out of this extracted information is not a simple task, requiring domain knowledge and creativity. However, machine learning techniques can be adopted to explore this information automatically and build new methods. One type of method which could assist in finding a new heuristic is the Hyper-Heuristic (HH). HH can be used to select or generate new heuristics specific to a problem, without much expertise given as input [12]. A Genetic Programming Hyper-Heuristic (GPHH) can create VRP solutions by generating a rule which selects the customers to be added into the current partial solution during the solution construction process. Genetic Programming (GP) is a type of Evolutionary Computation (EC) algorithm which produces a heuristic (or rule) based on trees of simple terminals related to the problem, and simple functions such as arithmetic operators. This rule can be more easily interpretable not only because of its tree representation but also by showing how the components impact the rule created, giving insight into how the solution is being built. Therefore, GPHH has been widely used to evolve heuristics for many complex problems, such as scheduling [13], VRP [14], Arc Routing [15], etc. However, the existing

GPHH work has shown relatively weak results when creating solutions, even when compared to traditional methods. One of the reasons for this is the lack of understanding of how the solution search space is.

This work proposes to utilise part of the aforementioned findings (from [9]), automatically. The idea is to let a GPHH evolve a rule which finds a solution situated in a more promising region of the solution search space. This is done by considering new terminals based on one of the metrics utilised in [9], *the width of the routes*, which is a measure of a route's compactness. Additionally, a new fitness function is proposed. This function minimises both the width and the overall distance in order to guide the solution to a more suitable region. The initial solution created by the GP-evolved rule is then fed into a variant of the KGLS for further improvement, aiming to reach a better final solution than when using a traditional savings heuristic [16] or even another GPHH without the width metric considered to initialise the solution. To cope with the large search space, a training/test approach is considered, where the training set is fed with smaller-scale instances.

The key contributions of this work are:

- New function terminals and a new fitness function which considers the width of a route on the route construction.
- Show that smaller instances can be used to train the GP rule and find success in larger instances.
- Verify that the GP-evolved heuristic can create an initial solution that can be successfully driven towards better final solutions in less time than traditional distance-based approaches.

II. Background

A. Large Scale Vehicle Routing Problem

The traditional Vehicle Routing Problem (VRP) was originally introduced by Dantzig in 1959 [17] and, since then is studied for its several applications in the real-world and for theoretical insight on how an NP-hard problem (as proven by [18]) can be solved efficiently and effectively. In literature, when the instances have at least 200 customers, they are considered large scale [1]–[4] and the problem is referred to as the Large Scale Vehicle Routing Problem (LSVRP). Although only an extension in the number of customers being served, the LSVRP face additional challenges due to its larger search space, which grows exponentially. Heuristic methods have been leading the solving of such scales through approximation techniques. Their goal is to find acceptable solutions in a reasonable time. The current state-of-the-art methods for the LSVRP are the Local Search-based heuristics, which explore different neighbourhoods (a set of solutions that are nearby in some sense, given a specific problem [19]).

To present a more formal definition of the Capacitated VRP (CVRP, which is the classical version of the problem) can exemplify why the problem is hard to solve. The CVRP (and, therefore, the LSVRP) consists of a number of customers which need to be served by a set of vehicles, with minimal cost. This is represented by a graph $G = \{V, E\}$, where V is the set of customers (or vertices in the graph, with a special node, referred to as depot) and E is the set of connections between them (or edges). Not only the number of possible connections is large, but there are additional restrictions that ensure the solution is feasible to applications, such as a limited capacity in all vehicles, the vehicles are required to start and end at the depot, and no customer may be visited more than once. More information on the problem can be found in the review of [20].

B. Related Work

Recent methods for the LSVRP have been focusing on efficient ways to prune the neighbourhood size and how to guide the search space towards a region where better solutions can be found. The works of [4], [9], [11] are examples of this. In these papers, the authors publish a study showing which solution-wise or instance-wise characteristics are more relevant when solving the VRP and LSVRP. They show that instance characteristics (such as customers position) are almost irrelevant when solving the problem, and the most impact can be seen in the solution itself. A few of the metrics they measured such as compactness of a route, average span, or average width, have shown to be more relevant. The average width was utilised to build a new penalisation criterium for a new Guided Local Search (GLS) algorithm, which guides the solution search space to the region where the routes have a smaller width.

The Knowledge-Guided Local Search (KGLS) [11] was introduced by adapting an existing classical heuristic, into a powerful VRP solver. These new guidelines to the search space have shown to be very effective, breaking several state-of-the-art methods, especially for large scale [4]. This method works as follows: from an initial solution (they use a modified version of the Savings heuristic [16]) the solution is improved until a local optimum. From this, there is a perturbation phase, where the solution is guided according to three different penalisation functions which are applied after each iteration. Then, another optimisation is applied, repeating these two phases until the stopping criteria. The algorithm is summarised in Algorithm 1. Equation 1 is utilised as the new objective to be optimised during the penalisation phase, which considers the penalised edges (with L being a parameter based on the Savings heuristics). Therefore, guiding the solution towards solutions that have a smaller width.

$$g(i,j) = c(i,j) + \lambda p(i,j)L \tag{1}$$

$$width(i,j) = max(d_{iE}, d_{jE}) - min(d_{iE}, d_{jE}), E = Line(D, G) \tag{2}$$

The width of a solution is a measure presented in their study [9], and calculates the distance to a line (E) that is traced from the depot (D) to the centre of gravity of a route (G), represented in Equation 2, where d_{iE} and d_{jE} are the distances to E. More details on the width are presented on the original paper [9]. Additionally, and especially for the large scale version (in [4]), several pruning techniques are applied to the

improvement heuristics used in their work (Cross-Exchange and Relocate-Chain).

Other works also deal with the large scale by focusing more on the improvement of the solution, pruning the neighbourhood size and the strategies to escape local optimum. Recent examples can be seen in the works of [21], where the neighbourhoods are reduced based on a distance or an angle basis between different routes, avoiding comparisons among the farther routes and customers based on a threshold. They generate random initial solutions and apply them to a Variable Neighbourhood Descent algorithm. Although no comparison to other methods is shown, they compare the effect of different thresholds and how they improve the execution time, without much loss of quality.

In [23] the authors present a hierarchical decomposition method, based on an Iterated Local Search framework. The customers in the LSVRP instance are divided into subgroups, and the goal becomes finding a permutation of tasks and then splitting them to find a solution, which is solved recursively. Then the solution is given to the adapted Iterated Local Search (ILS) procedure which improves the solution with Local Search which are then fed to a new hierarchical decomposition scheme. The results, however, are not competitive, not even beating the traditional Savings algorithm for some instances.

An evolutionary multi-objective approach based on route grouping was proposed in [24]. The authors considered a divide-and-conquer to do the decomposition based on three distinct objectives: intra-group distance, inter-group distances and balance in size. These objectives are considered to find better clusters which are then optimised with a Tabu Search algorithm. They show that their multi-objective approach outperforms another considering the K-means to create the groups. Overall, the method shows competitive results with execution times of less than 1 hour for instances of up to 1200 customers.

However, the success of these methods is bound to the quality of the heuristic pruning methodology applied to it. This requires not only knowledge about the data being processed, but also about what the VRP neighbourhoods look like and represent. To minimise this domain knowledge necessary for finding good heuristics, Hyper-heuristics were proposed [12]. Their goal is to let the machine learn what the heuristic should be like, and what part of the data is necessary for efficiently solving the problem. One of the main methods for heuristic automation is Genetic Programming.

C. GPHH for VRP

Genetic Programming (GP) is an Evolutionary Computation (EC) technique in which rules and programs are evolved. The GP usually utilises a tree-based data-structure where each leaf represents a terminal (a variable or fixed value), while the internal nodes represent functions (operators which compute the leaves) [25]. When this tree is used to create a new heuristic rather than a solution, it can be referred to as Genetic Programming Hyper-Heuristic (GPHH). A recent review of GP as Hyper-Heuristic can be found in [26].

GPHH have successfully been used for solving combinatorial optimisation problems, such as Job Shop Scheduling [27], Arc Routing Problem [15], [28]. However, the most successful applications are for Dynamic or Uncertain variants, as GPHH provides a rule to quickly determine the new stage of an existing solution. For the static problems, GPHH has been applied but are less competitive. For example, for solving the VRP with Time Windows (VRPTW), the work of [29] have applied a GP for building an initial solution, followed by a selection hyper-heuristic for improvement. Their GP, even though has several novel terminals added, still produces fairly weak results.

In [30], the authors use a grammar-based GP, known as Grammatical Evolution (GE). GE uses strings expressed in *Backus Naur Form* which are translated into the resulting program. The pre-defined grammar translates these strings, having several steps such as finding an initial solution, ruining and recreating rules, and terminal points and functions where the routes' components and functions are stored, respectively. The resulting heuristic is used to feed a Variable Neighbour Search (VNS) system to evolve a solution. The work, however, explores very little on the results, not finding competitive results even for small scale instances (less than 40 customers for the CVRP).

The work of [31] proposes a GE method for the CVRP where the resulting string also contains the calls for improving the solution. One of the possible improvement moves is a deterministic variation of the ILS. The grammar consists of 4 main elements which are applied consecutively. First, a strategy is defined to create an initial solution (or partial solution). Then, constructive operators are utilised to build a feasible solution. The third phase will improve the solution. And, finally, a fourth phase where the process is repeated in several cycles to find the final improved solution. They found the best results for most of the 40 instances tested, of up to 101 customers, even improving some of them.

These are examples of how GP still fails to build competitive solutions, and why the field is dominated by improvement-based heuristics, which was incorporated in the Grammatical-based ones. Although this lack of near-optimal solutions is the case for most GPHH-based methods, this work shows that by considering the correct elements, a GPHH can at least guide this initial solution towards a better region of the search space.

III. GPHH for Effective Initialisation

The idea for the proposed GPHH is mainly based on evolving a rule which guides the construction of an LSVRP solution towards a more promising region of the search space. This is done by considering one solution characteristic, the width of a route, which was shown to be one of the main common metrics to the closeness to optimal solutions, as shown in [9]. The proposed GPHH utilises the same tree-based representation as most GPHH do (such as in [13]–[15], [28], [29]). The GPHH utilises a terminal and a function set to build the routing policy given a fitness function, which undergoes an evolutionary process to learn how to do so. The novelty

Acronym	Description
DFD	Distance from considered customer to the depot
DEM	Demand of considered customer
DFCP	Distance from considered customer to the last customer added to the route
AVGD	Average distance from remaining non-served customers to the customer considered
AVGP	Average distance from remaining non-served customers to the last customer added to the route
CWD	Clarke and Wright [16] savings distance
MCDC	Most constrained demand
WINACR	Width of the current route after adding considered node
WICR	Width of the current route
AVGWI	Average width of all routes

TABLE I: Set of terminals for the GPHH

of this algorithm takes place on the new terminal set and the fitness function. For the terminal, 3 new ones are proposed, as follows:

- WINACR - The width of the current position after adding the node considered. This terminal calculates the width of the current route when adding a new customer. Equation 2 is used, by considering the last customer from a route (i) and the candidate customer (j).
- WICR - The average width of the route being considered. This terminal calculates the average width of the current route evaluated. Equation 3 is used for the route considered.
- AVGWI - The average width of all routes in the solution. Calculated with Equation 4.

These terminals are similar to the traditional ones already utilised in the construction of VRP solutions. The main difference being the metric considered now is the width of the routes. By adding these, the GPHH now has information about this metric and can evolve to create solutions that take this feature into account. The set of all terminal nodes are presented in Table I.

The second novelty, the fitness function (Equation 4), also takes into account the width metric. As finding a competitive solution as the initial one is very unlikely, the primary goal of this complex optimisation problem is to let the improvement operators guide the solution towards the best final solution. However, if incorporating the full improvement run into the GPHH training, the required time would be too expensive. In other words, if the KGLS* (a simpler version of the KGLS which uses less complex improvement methods), as shown in Algorithm 1, were to be executed during the evaluation of each individual. To make this more efficient, a surrogate objective function is proposed. The function N_p^t calculates the normalised distance of the route, given individual p, and also the average width of all routes, for the instance t, which is calculated as shown in Equation 3. The normalization is in comparison to the Best-Known-Solution (BKS).

$$AvgWidth(t) = \frac{\sum_{(i,j)\in E} width(i,j)}{||E||} \tag{3}$$

$$N_p^t = \frac{\sum_{(j,k)\in E} d_{jk}x_{jk}}{BKS^t} + AvgWidth(t) \tag{4}$$

Algorithm 1: Knowledge-Guided Local Search used to update the solution.

```
Input  : Initial Solution S, Instance i, Number of penalised moves P
Output : Final Solution S*
while Local Optimum not reached do
    foreach Intra-route Improvemet Heuristic do
        Apply Heuristic to the current solution with objective f ;
        Save set of changed routes
while Local Optimum not reached do
    foreach Inter-route Improvemet Heuristic do
        Apply Heuristic to the routes which were changed in the
          previous step, with objective f ;
while Stopping criteria not met do
    while Number of penalised moves ≤ P do
        Penalise current solution with current penalise function b_j ;
        Select penalised edge e ;
        foreach Improvemet Heuristic do
            Apply Heuristic to the current solution with objective
              g ;
        if A move was made then
            Increment number of penalised moves ;
    while Local Optimum not reached do
        foreach Improvemet Heuristic do
            Apply Heuristic to the current solution with objective
              f ;
    Change penalisation function b to the next one ;
Return value of best solution (f(x)) ;
```

A. Route Construction and Evaluation

The route construction algorithm is used to build the solution considering the current GP tree and instance. As shown in Algorithm 2, the procedure builds the routes simultaneously. The rule from the GP-tree ranks the routes in which the customer can be added. Therefore, the terminals are calculated for each route as well as the customer being added. All customers are evaluated by their index sequence and the target route is selected according to the GP-tree.

The final solution is then evaluated with Equation 4. The individual (p) of the population will be evaluated by considering the sum for all instances, as shown in Equation 5. And since we only normalize the distance, the width will play a more important role when minimising the fitness function.

$$F_p = \sum_{t\in Instances} N_p^t \tag{5}$$

The algorithm tries to add all customers with a minimal number of routes. The initial number of routes is an approximation considering the capacity and total demand. The customers are ranked one at a time for each route. The highest-ranked route will have the customer added there.

B. Genetic Operators

As the focus of this work is more related to validate the new terminals and fitness function, the genetic operators are chosen from traditional methods, with a limited depth for avoiding very large trees. The overall algorithm for the GP training can be seen in Algorithm 3. First, the population is created at random. Then, the population is evaluated according to the methods mentioned beforehand. The select method is a

Algorithm 2: Route construction method.

```
Input  : GP Tree T, Instance I
Output : Built Solution S
Number of routes = ⌈ TotalDemand / VehicleCapacity ⌉ ;
foreach Customer c ∈ I do
    foreach Vehicle v do
        Apply function tree T(c, v) ;
    Calculate and Select highest ranked vehicle ;
    if No vehicle is feasible then
        Add new vehicle ;
        Select new vehicle ;
    S = Add customer c to selected vehicle's route last position ;
Return Built solution S ;
```

Algorithm 3: Genetic Programming training method.

```
Input  : Training Set T
Output : Best Tree S
Initialise Population at random ;
while Number of Generations not reached do
    foreach Individual i ∈ Population do
        foreach Instance t ∈ T do
            Evaluate i with given t by Equation 4 ;
        Compute fitness of i by Equation 5 ;
    Select Parents ;
    Crossover ;
    Mutation ;
Return Best individual S ;
```

Tournament Selection. The best individual is always kept in the population, unchanged. The crossover is a one-point crossover, where sub-trees are exchanged from the selected parents. For mutation, a new random sub-tree is added in the place of one of the leaves. Both crossover and mutation operators are prune to not overflow the maximum depth of the tree.

IV. Experiment Design

The experiments were done using the dataset presented in [32]. This dataset presents different types of customer distribution, demands, capacity and depot location. The instances range from 100 to 1000 customers. The dataset was split into training and test sets. The training set includes all the instances with less than 200 customers, while the test set includes all the remaining instances.

To fully evaluate the performance of the proposed operators and that they are indeed guiding into a better region of the search space, the experiments were designed in a way to validate this. The solution created with the proposed GPHH is expected to be improved quickly into a better final solution. To measure this, the solutions created with the proposed GPHH approach are improved by the KGLS* algorithm, as shown in Algorithm 1.

The KGLS uses the parameters in Table II and utilised the following set of intra and inter (bold) route moves:

- Swap: Swaps the position of a pair of customers within the same route.
- Two Opt: Removes two edges and adds two new ones within the same route.
- Three Opt: Removes three edges and add three new ones within the same routes.

Parameter	Value
Execution time	10 min
Number of Penalised Moves (P)	10
Number of Closest Customers	30
lambda (λ)	0.1

TABLE II: Knowledge-Guided Local Search parameters.

Parameter	Value
Population size	1000
No. of Generations	50
Cross. Rate	0.8
Mut. Rate	0.2
Max depth	8
Tournament Size	7
Number of runs	15

TABLE III: Genetic Programming parameters.

- **Cross Exchange**: Exchanges a pair of sub-routes of size k between two routes.
- **Swap Star**: Swaps the position of a pair of customers on different routes.
- **Relocate**: Relocates a customer to a different route.

Another GPHH (GPHH^2) was also trained to compare the effect of the new terminals. The GPHH^2 utilises the same function set (Table I) and fitness function (Equation 5) as the proposed GPHH. The only difference is that it is trained without any of the width-based terminals. Therefore, this GPHH^2 can measure whether these terminals are influential in finding a better solution. The compared GPHH methods use the parameters present in Table III.

Two main experiments were realized. The first experiment compares the proposed GPHH as the starting point versus the traditional Savings initialisation heuristic [16], when fed into the KGLS*. Similar to the previous experiment, the best individual from the proposed GPHH will create the initial solution for the KGLS and measure how much the final solution was improved, in a fixed amount of time (10 minutes in these experiments). Additionally, the methods are compared on how fast do they improve the solution. The solution is also compared to the original KGLS, captured from the available online provided[1], also run for the 10 minutes.

The second experiment compares the proposed GPHH versus the GPHH^2, to validate the effectiveness of the newly designed width-related terminals in helping GP finding better initialisation heuristics. In this experiment, the best individuals of both GPHHs will create an initial solution for the KGLS* (Algorithm 1) and be run for the same amount of time (10 minutes).

V. Results and Discussions

Impact of initial solution: To measure how much the initial solution influenced the evolution, the experiment was done to compare how the final solution was improved with the same amount of time. Table IV compile these results in comparison to the Best-Known-Solution (BKS)[2].

[1]Available in the website: https://antor.uantwerpen.be/routingsolver/

[2]Values gathered from the dataset website: http://vrp.atd-lab.inf.puc-rio.br/index.php/en/

A statistical test (Wilcoxon Rank-Sum test with a significance value of 0.05) was performed to validate the proposed method's performance when compared to the KGLS* with the Savings as initialisation heuristic. Table IV highlights the cases where there was an advantage for the GPHH. The values in boldface mean that there is a significant advantage between the proposed GPHH or the KGLS*. The best values found are also highlighted when the GPHH achieves that. In total, the proposed GPHH was able to have a significant advantage in 8 out of the 79 instances, and no advantage was found in other 23 instances. In other words, for about 40% of the instances the proposed method was just as good or better on the average case for all runs. Even though the initial solution has a much worse value when compared to the Savings heuristic, as can be seen in Figure 1. Additionally, when analysing the best-case performance, the proposed GPHH show very promising results. In this case, the proposed starting solution was even able to beat the state-of-the-art for 4 different instances, using simpler heuristics. It also beats the KGLS* on average. Figure 1 also shows the performance of the best-case, which starts badly but outperforms the KGLS* in the end.

Impact of proposed terminals: This test measures if the GP novel terminals have any influence on the quality observed. Surprisingly, by comparing to the GPHH^2 (which only uses the objective function)the results show that the fitness function is the major contributor to the quality achieved. As can be seen in Table V, the GPHH^2 outperforms the version with the terminals. This can be an indication that the extra terminals increase the search space for the GP evolutionary process, making the search more difficult. Whereas just the fitness function is enough to guide the solution towards a region of the search space which can be more quickly improved.

In fact, when comparing to the KGLS*, the GPHH^2 converges much faster than the proposed GPHH with the terminals. As shown in Figure 2, the time required for the GPHH^2 initial solution to surpass the KGLS* is less than half of that of the proposed GPHH. And the improvement for the best case in on average 1.39% at the end of execution. Therefore, the new fitness function is the main reason for the quality of the final solution presented.

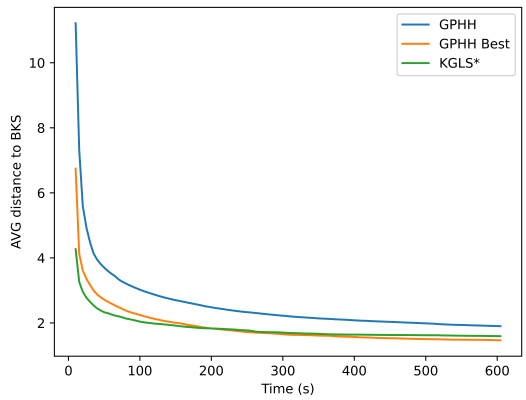

Fig. 1: Comparison between the proposed GPHH average and best case to the KGLS* performance.

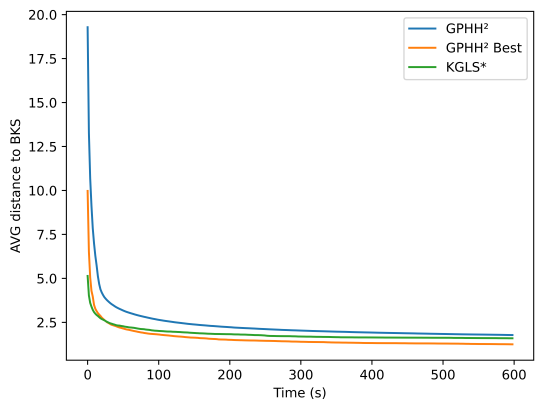

Fig. 2: Comparison between the GPHH^2 average and best case to the KGLS* performance.

VI. Conclusions

In this work, a Genetic Programming Hyper-Heuristic is proposed to construct LSVRP solutions in a specific region of the search space. The new proposed fitness function guides the solutions towards routes with less width, which have shown to be a good indicator of near-optimal solutions. The method is able to produce a solution that can be improved towards better final solutions compared to a traditional heuristic method in 49 out of the 79 test instances. Additionally, the proposed GPHH (improved by the KGLS*) was able to achieve the best solution in the presented tests for a few instances, even when using simpler heuristics than the state-of-the-art.

Although the new proposed terminals have shown to be less effective towards the goal of improving the solutions, the overall results still show the effectiveness of the proposed GPHH successfully guiding towards a different region of the search space which is more promising. When considering the time in which the solution outperforms the KGLS* (in the best-case), as shown in Figures 1 and 2, GPHH can also be seen as more efficient. Especially when considering that this is achieved just by changing the starting point.

For the future work, a more detailed analysis can be made. Testing the built solution with other meta-heuristics can show whether the improvement observed in this work is not bound to the KGLS. A more competitive and efficient route construction method can be considered. And new terminals and/or a new fitness function with additional metrics could also be considered, using the power of GP to learn how to use other solution characteristics automatically.

References

[1] M. Gendreau and C. D. Tarantilis, *Solving large-scale vehicle routing problems with time windows: The state-of-the-art*. Cirrelt Montreal, 2010.

TABLE IV: Results compared to the Best Known Solution (BKS). In bold show significance in the comparison between the KGLS* and our GPHH. Underlined are the overall best.

Instance	KGLS to BKS	KGLS* to BKS	Proposed GPHH Avg. to BKS	GPHH Best to BKS	Instance	KGLS to BKS	KGLS* to BKS	Proposed GPHH Avg. to BKS	GPHH Best to BKS
X-n200-k36	0.29%	2.28%	1.70%± 0.63%	**0.84%**	X-n429-k61	0.43%	**1.58%**	1.77%± 0.11%	1.54%
X-n204-k19	0.52%	1.05%	1.16%± 0.19%	**0.35%**	X-n439-k37	0.39%	0.93%	0.96%± 0.14%	**0.71%**
X-n209-k16	0.25%	1.69%	1.68%± 0.29%	**0.62%**	X-n449-k29	0.75%	**2.55%**	2.87%± 0.23%	**2.44%**
X-n214-k11	0.67%	2.46%	3.78%± 1.70%	**1.75%**	X-n459-k26	0.20%	**1.80%**	2.33%± 0.32%	1.84%
X-n219-k73	0.07%	0.15%	0.15%± 0.02%	**0.07%**	X-n469-k138	0.66%	1.20%	**1.11%**± 0.08%	**1.06%**
X-n223-k34	0.59%	1.79%	**1.55%**± 0.18%	**1.26%**	X-n480-k70	0.38%	**1.27%**	1.60%± 0.21%	1.28%
X-n228-k23	0.37%	1.37%	1.90%± 0.65%	**1.13%**	X-n491-k59	0.75%	**1.78%**	2.03%± 0.18%	1.90%
X-n233-k16	0.54%	**1.12%**	1.47%± 0.33%	**0.93%**	X-n502-k39	0.11%	**0.57%**	0.63%± 0.06%	0.58%
X-n237-k14	0.24%	**0.82%**	1.29%± 0.55%	**0.29%**	X-n513-k21	0.44%	**2.16%**	2.72%± 0.39%	**2.05%**
X-n242-k48	0.47%	**1.23%**	1.34%± 0.15%	**0.72%**	X-n524-k137	1.77%	2.66%	**2.44%**± 0.47%	**1.82%**
X-n247-k47	1.11%	2.60%	2.06%± 0.36%	**1.34%**	X-n536-k96	0.81%	1.55%	**1.45%**± 0.19%	**1.22%**
X-n251-k28	0.60%	1.64%	1.61%± 0.15%	**1.01%**	X-n548-k50	0.25%	1.00%	**0.91%**± 0.09%	**0.83%**
X-n256-k16	0.05%	0.80%	1.58%± 1.01%	**0.75%**	X-n561-k42	0.63%	**1.71%**	2.24%± 0.26%	1.75%
X-n261-k13	0.43%	2.00%	2.53%± 0.97%	**1.41%**	X-n573-k30	0.19%	**1.39%**	2.36%± 0.47%	2.04%
X-n266-k58	0.64%	1.31%	1.33%± 0.12%	**0.91%**	X-n586-k159	0.53%	1.19%	1.08%± 0.14%	**0.94%**
X-n270-k35	0.45%	**0.95%**	1.10%± 0.12%	0.95%	X-n599-k92	0.54%	**1.22%**	1.35%± 0.15%	1.30%
X-n275-k28	0.16%	**0.97%**	1.17%± 0.16%	**0.81%**	X-n613-k62	0.72%	**2.27%**	2.51%± 0.26%	2.33%
X-n280-k17	0.56%	**2.28%**	3.59%± 0.72%	2.96%	X-n627-k43	0.31%	**1.78%**	2.80%± 0.61%	1.99%
X-n284-k15	0.80%	**2.31%**	2.92%± 0.52%	**2.25%**	X-n641-k35	0.39%	**1.90%**	3.18%± 0.52%	2.40%
X-n289-k60	0.86%	1.56%	1.63%± 0.17%	**1.16%**	X-n655-k131	0.19%	0.49%	0.51%± 0.04%	**0.45%**
X-n294-k50	0.41%	1.65%	1.61%± 0.17%	**0.97%**	X-n670-k126	3.28%	5.33%	**2.68%**± 0.59%	**1.94%**
X-n298-k31	0.41%	**1.34%**	2.02%± 0.37%	1.55%	X-n685-k75	0.70%	**1.72%**	2.05%± 0.21%	**1.62%**
X-n303-k21	0.61%	**1.72%**	2.05%± 0.45%	2.06%	X-n701-k44	0.22%	**1.55%**	2.24%± 0.25%	1.73%
X-n308-k13	1.17%	**2.39%**	3.97%± 0.96%	**1.64%**	X-n716-k35	0.69%	**2.60%**	3.44%± 0.49%	3.48%
X-n313-k71	0.93%	**1.49%**	1.61%± 0.13%	**1.19%**	X-n733-k159	0.61%	**1.39%**	1.65%± 0.14%	1.43%
X-n317-k53	0.07%	**0.41%**	0.66%± 0.15%	**0.28%**	X-n749-k98	0.58%	**1.69%**	2.14%± 0.30%	1.75%
X-n322-k28	0.58%	1.94%	1.86%± 0.17%	**1.53%**	X-n766-k71	1.04%	**2.84%**	5.65%± 0.91%	4.34%
X-n327-k20	0.40%	**1.33%**	1.82%± 0.38%	1.43%	X-n783-k48	0.49%	**1.60%**	4.03%± 0.54%	3.37%
X-n331-k15	0.24%	**1.28%**	1.71%± 0.45%	**1.14%**	X-n801-k40	0.07%	**0.96%**	3.14%± 0.84%	1.62%
X-n336-k84	1.03%	1.66%	1.82%± 0.18%	**1.42%**	X-n819-k171	0.61%	**1.16%**	1.58%± 0.22%	1.37%
X-n344-k43	0.71%	**1.57%**	1.71%± 0.16%	**1.43%**	X-n837-k142	0.46%	1.26%	1.37%± 0.18%	**1.15%**
X-n351-k40	0.82%	2.58%	2.56%± 0.33%	**1.52%**	X-n856-k95	0.34%	**0.83%**	0.99%± 0.13%	**0.73%**
X-n359-k29	0.93%	**1.52%**	2.11%± 0.22%	**1.42%**	X-n876-k59	0.42%	**1.67%**	1.95%± 0.23%	1.86%
X-n367-k17	0.51%	**2.04%**	3.63%± 0.63%	**1.43%**	X-n895-k37	0.20%	**2.54%**	4.53%± 0.90%	3.26%
X-n376-k94	0.09%	0.32%	0.30%± 0.04%	**0.24%**	X-n916-k207	0.43%	**0.91%**	1.30%± 0.14%	1.15%
X-n384-k52	0.44%	1.39%	1.31%± 0.16%	**0.89%**	X-n936-k151	3.40%	6.22%	**3.79%**± 0.94%	**2.97%**
X-n393-k38	0.53%	1.93%	1.99%± 0.24%	**1.42%**	X-n957-k87	0.47%	**0.84%**	1.35%± 0.22%	1.05%
X-n401-k29	0.54%	**1.05%**	1.56%± 0.58%	**0.81%**	X-n979-k58	0.55%	**1.23%**	4.03%± 1.75%	1.92%
X-n411-k19	1.89%	**2.70%**	4.94%± 1.44%	4.81%	X-n1001-k43	0.40%	**2.85%**	4.06%± 0.50%	3.44%
X-n420-k130	0.56%	1.35%	**1.18%**± 0.11%	**0.71%**	**Average**	0.61%	1.54%	2.09%	1.64%

[2] M. Huang and X. Hu, "Large scale vehicle routing problem: An overview of algorithms and an intelligent procedure," *International Journal of Innovative Computing, Information and Control*, vol. 8, no. 8, p. 5809–5819, 2012.

[3] N. R. Sabar, X. J. Zhang, and A. Song, "A math-hyper-heuristic approach for large-scale vehicle routing problems with time windows," in *CEC*. IEEE, 2015, p. 830–837.

[4] F. Arnold, M. Gendreau, and K. Sörensen, "Efficiently solving very large-scale routing problems." *Computers & OR*, vol. 107, p. 32–42, 2019.

[5] B. Hollis, M. Forbes, and B. Douglas, "Vehicle routing and crew scheduling for metropolitan mail distribution at australia post," *European Journal of Operational Research*, vol. 173, no. 1, p. 133–150, 2006.

[6] K. Buhrkal, A. Larsen, and S. Ropke, "The waste collection vehicle routing problem with time windows in a city logistics context," *Procedia - Social and Behavioral Sciences*, vol. 39, p. 241–254, 2012, seventh International Conference on City Logistics which was held on June 7-9,2011, Mallorca, Spain.

[7] T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, "A unified solution framework for multi-attribute vehicle routing problems," *European Journal of Operational Research*, vol. 234, no. 3, p. 658–673, may 2014.

[8] A. Subramanian, E. Uchoa, and L. S. Ochi, "A hybrid algorithm for a class of vehicle routing problems," *Computers & Operations Research*, vol. 40, no. 10, p. 2519–2531, oct 2013.

[9] F. Arnold and K. Sörensen, "What makes a vrp solution good? the generation of problem-specific knowledge for heuristics," *Computers & Operations Research*, vol. 106, p. 280–288, 2019.

[10] F. Lucas, R. Billot, and M. Sevaux, "A comment on "what makes a vrp solution good? the generation of problem-specific knowledge for heuristics"," *Computers & Operations Research*, vol. 110, p. 130–134, 2019.

[11] F. Arnold and K. Sörensen, "Knowledge-guided local search for the vehicle routing problem," *Computers & Operations Research*, vol. 105, p. 32–46, may 2019.

[12] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art," *Journal of the Operational Research Society*, vol. 64, no. 12, p. 1695–1724, 2013.

[13] F. Zhang, Y. Mei, and M. Zhang, "A two-stage genetic programming hyper-heuristic approach with feature selection for dynamic flexible job shop scheduling," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, p. 347–355.

[14] J. Jacobsen-Grocott, Y. Mei, G. Chen, and M. Zhang, "Evolving heuristics for dynamic vehicle routing with time windows using genetic programming," in *2017 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2017, p. 1948–1955.

[15] M. A. Ardeh, Y. Mei, and M. Zhang, "Transfer learning in genetic programming hyper-heuristic for solving uncertain capacitated arc routing problem." in *CEC*. IEEE, 2019, p. 49–56.

[16] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot

TABLE V: Results compared to the Best Known Solution (BKS) for the two compared GPHHs.

Instance	**Proposed GPHH Avg. to BKS**	**Proposed GPHH^2 Avg. to BKS**	**Instance**	**Proposed GPHH Avg. to BKS**	**Proposed GPHH^2 Avg. to BKS**
X-n200-k36	1.70%	1.89%	X-n429-k61	1.77%	1.66%
X-n204-k19	1.16%	1.03%	X-n439-k37	0.96%	0.92%
X-n209-k16	1.68%	1.64%	X-n449-k29	2.87%	2.71%
X-n214-k11	3.78%	2.55%	X-n459-k26	2.33%	2.47%
X-n219-k73	0.15%	0.14%	X-n469-k138	1.11%	1.12%
X-n223-k34	1.55%	1.56%	X-n480-k70	1.60%	1.51%
X-n228-k23	1.90%	1.68%	X-n491-k59	2.03%	2.07%
X-n233-k16	1.47%	1.33%	X-n502-k39	0.63%	0.58%
X-n237-k14	1.29%	1.24%	X-n513-k21	2.72%	2.36%
X-n242-k48	1.34%	1.35%	X-n524-k137	2.44%	2.55%
X-n247-k47	2.06%	2.49%	X-n536-k96	1.45%	1.45%
X-n251-k28	1.61%	1.50%	X-n548-k50	0.91%	0.88%
X-n256-k16	1.58%	1.34%	X-n561-k42	2.24%	2.15%
X-n261-k13	2.53%	2.49%	X-n573-k30	2.36%	1.78%
X-n266-k58	1.33%	1.29%	X-n586-k159	1.08%	1.11%
X-n270-k35	1.10%	1.00%	X-n599-k92	1.35%	1.47%
X-n275-k28	1.17%	1.01%	X-n613-k62	2.51%	2.39%
X-n280-k17	3.59%	2.57%	X-n627-k43	2.80%	2.11%
X-n284-k15	2.92%	2.43%	X-n641-k35	3.18%	2.60%
X-n289-k60	1.63%	1.67%	X-n655-k131	0.51%	0.52%
X-n294-k50	1.61%	1.65%	X-n670-k126	2.68%	2.60%
X-n298-k31	2.02%	2.17%	X-n685-k75	2.05%	2.10%
X-n303-k21	2.05%	2.16%	X-n701-k44	2.24%	1.95%
X-n308-k13	3.97%	4.18%	X-n716-k35	3.44%	2.80%
X-n313-k71	1.61%	1.66%	X-n733-k159	1.65%	1.58%
X-n317-k53	0.66%	0.50%	X-n749-k98	2.14%	2.08%
X-n322-k28	1.86%	1.74%	X-n766-k71	5.65%	4.42%
X-n327-k20	1.82%	1.90%	X-n783-k48	4.03%	3.59%
X-n331-k15	1.71%	1.54%	X-n801-k40	3.14%	2.10%
X-n336-k84	1.82%	1.82%	X-n819-k171	1.58%	1.54%
X-n344-k43	1.71%	1.72%	X-n837-k142	1.37%	1.41%
X-n351-k40	2.56%	2.38%	X-n856-k95	0.99%	1.07%
X-n359-k29	2.11%	1.98%	X-n876-k59	1.95%	1.79%
X-n367-k17	3.63%	3.20%	X-n895-k37	4.53%	3.62%
X-n376-k94	0.30%	0.29%	X-n916-k207	1.30%	1.35%
X-n384-k52	1.31%	1.41%	X-n936-k151	3.79%	3.28%
X-n393-k38	1.99%	2.01%	X-n957-k87	1.35%	1.15%
X-n401-k29	1.56%	2.05%	X-n979-k58	4.03%	2.44%
X-n411-k19	4.94%	4.27%	X-n1001-k43	4.06%	3.57%
X-n420-k130	1.18%	1.14%	**Average**	2.09%	**1.91%**

to a number of delivery points," *Operations research*, vol. 12, no. 4, p. 568–581, 1964.

[17] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, p. 80–91, oct 1959.

[18] J. K. Lenstra and A. H. G. R. Kan, "Complexity of vehicle routing and scheduling problems." *Networks*, vol. 11, no. 2, p. 221–227, 1981.

[19] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.

[20] G. Laporte, "Fifty years of vehicle routing," *Transportation Science*, vol. 43, no. 4, p. 408–416, nov 2009.

[21] H. Liu, Z. Zhang, and X. Guo, "Restricted neighborhood search for large scale vehicle routing problems," in *2019 IEEE 16th International Conference on Networking, Sensing and Control (ICNSC)*. IEEE, may 2019.

[22] Komarudin and A. Chandra, "Optimization of very large scale capacitated vehicle routing problems," in *Proceedings of the 2019 5th International Conference on Industrial and Business Engineering - ICIBE 2019*. ACM Press, 2019.

[23] E. Zhuo, Y. Deng, Z. Su, P. Yang, B. Yuan, and X. Yao, "An experimental study of large-scale capacitated vehicle routing problems," 2019.

[24] J. Xiao, T. Zhang, J. Du, and X. Zhang, "An evolutionary multiobjective route grouping-based heuristic algorithm for large-scale capacitated vehicle routing problems," *IEEE transactions on cybernetics*, p. 1–14, 2019.

[25] W. B. Langdon, R. I. McKay, and L. Spector, "Genetic programming," in *Handbook of metaheuristics*. Springer, 2010, p. 185–225.

[26] H. Al-Sahaf, Y. Bi, Q. Chen, A. Lensen, Y. Mei, Y. Sun, B. Tran, B. Xue, and M. Zhang, "A survey on evolutionary machine learning," *J. Roy. Soc. New Zeal.*, vol. 49, no. 2, p. 205–228, 2019.

[27] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job shop scheduling," *IEEE Transactions on Cybernetics*, 2020. Doi: 10.1109/TCYB.2020.3024849.

[28] S. Wang, Y. Mei, and M. Zhang, "A multi-objective genetic programming hyper-heuristic approach to uncertain capacitated arc routing problems." in *CEC*. IEEE, 2020, p. 1–8.

[29] K. Sim and E. Hart, "A combined generative and selective hyper-heuristic for the vehicle routing problem." in *GECCO*, T. Friedrich, F. Neumann, and A. M. Sutton, Eds. ACM, 2016, p. 1093–1100.

[30] J. H. Drake, N. Kililis, and E. Özcan, "Generation of VNS components with grammatical evolution for vehicle routing," in *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 2013, p. 25–36.

[31] R. J. Marshall, M. Johnston, and M. Zhang, "Developing a hyper-heuristic using grammatical evolution and the capacitated vehicle routing problem." in *SEAL*, ser. Lecture Notes in Computer Science, vol. 8886. Springer, 2014, p. 668–679.

[32] E. Uchoa, D. Pecin, A. A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New benchmark instances for the capacitated vehicle routing problem." *Eur. J. Oper. Res.*, vol. 257, no. 3, p. 845–858, 2017.